IB Computer Science Python Syntax Error Log (2027 Syllabus)

Table of Contents

1. Python Basics
2. Data Types and Variables
3. Input/Output Operations
4. Control Structures
5. Functions and Scope
6. Data Structures
7. File Handling
8. Object-Oriented Programming
9. Error Handling and Exceptions
10. Modules and Libraries
11. Algorithm Implementation
12. Data Processing and Analysis
13. Network Programming
14. Database Operations
15. Advanced Python Features (2027 Additions)

Python Basics

Error Type: Indentation Errors

❌ **Incorrect:**

```python
if x > 0:
print("Positive")
    print("Greater than zero")
```

✅ **Correct:**

```python
```

```
if x > 0:
    print("Positive")
    print("Greater than zero")
```

Issue: Inconsistent indentation (Python uses 4 spaces as standard)

Error Type: Missing Colons

❌ **Incorrect:**

```
python

if age >= 18
    print("Adult")
```

✅ **Correct:**

```
python

if age >= 18:
    print("Adult")
```

Issue: Missing colon after conditional statement

Error Type: Case Sensitivity

❌ **Incorrect:**

```
python

Print("Hello World")
TRUE = 1
```

✅ **Correct:**

```
python

print("Hello World")
true_value = 1  # or use True (boolean)
```

Issue: Python keywords and functions are case-sensitive

Error Type: Mixing Tabs and Spaces

❌ **Incorrect:**

```python
def calculate():
    x = 5
  return x  # This line uses tab
```

✅ **Correct:**

```python
def calculate():
    x = 5
    return x  # Use consistent spaces
```

Issue: Mixing tabs and spaces causes IndentationError

Data Types and Variables

Error Type: Undefined Variables

❌ **Incorrect:**

```python
print(name)
name = "Alice"
```

✅ **Correct:**

```python
name = "Alice"
print(name)
```

Issue: Using variable before definition

Error Type: Type Conversion Errors

❌ **Incorrect:**

```python
age = input("Enter age: ")
if age > 18:
    print("Adult")
```

✅ **Correct:**

```python
age = int(input("Enter age: "))
if age > 18:
    print("Adult")
```

Issue: input() returns string; need conversion for numeric comparison

Error Type: String Indexing Errors

❌ **Incorrect:**

```python
text = "Hello"
text[0] = "h"  # Trying to modify string
```

✅ **Correct:**

```python
text = "Hello"
text = "h" + text[1:]  # Create new string
# or
text_list = list(text)
text_list[0] = "h"
text = "".join(text_list)
```

Issue: Strings are immutable in Python

Error Type: Division by Zero

❌ **Incorrect:**

```python
```

```
result = 10 / 0
```

✅ **Correct:**

```python
divisor = 0
if divisor != 0:
    result = 10 / divisor
else:
    print("Cannot divide by zero")
```

Issue: Division by zero raises ZeroDivisionError

Error Type: Integer Division Confusion

❌ **Incorrect:**

```python
# Python 3 - expecting integer result
result = 7 / 2  # Returns 3.5, not 3
```

✅ **Correct:**

```python
result = 7 // 2  # Returns 3 (floor division)
# or
result = int(7 / 2)  # Convert to integer
```

Issue: In Python 3, / always returns float

Input/Output Operations

Error Type: Print Function Syntax (Python 3)

❌ **Incorrect:**

```python
print "Hello World"  # Python 2 syntax
```

✅ **Correct:**

```python
print("Hello World")  # Python 3 syntax
```

Issue: Print is a function in Python 3, requires parentheses

Error Type: Input Type Handling

❌ **Incorrect:**

```python
numbers = input("Enter numbers: ")
total = sum(numbers)
```

✅ **Correct:**

```python
numbers = input("Enter numbers separated by space: ").split()
numbers = [int(x) for x in numbers]
total = sum(numbers)
```

Issue: input() returns string, need to process for numeric operations

Error Type: String Formatting Errors

❌ **Incorrect:**

```python
name = "Alice"
age = 25
print("Name: " + name + " Age: " + age)  # Type error
```

✅ **Correct:**

```python
```

```
name = "Alice"
age = 25
print("Name: " + name + " Age: " + str(age))
# or better:
print(f"Name: {name} Age: {age}")  # f-string
```

Issue: Cannot concatenate string and integer

Error Type: File Output Errors

❌ **Incorrect:**

```
python

print("Data", file="output.txt")
```

✅ **Correct:**

```
python

with open("output.txt", "w") as file:
    print("Data", file=file)
```

Issue: file parameter needs file object, not filename

Control Structures

Error Type: Assignment vs Comparison

❌ **Incorrect:**

```
python

if x = 5:  # Assignment instead of comparison
    print("x is 5")
```

✅ **Correct:**

```
python

if x == 5:  # Comparison operator
    print("x is 5")
```

Issue: Using = (assignment) instead of == (comparison)

Error Type: Boolean Logic Errors

❌ **Incorrect:**

```python
if x > 0 && x < 10:  # C-style operators
    print("Valid range")
```

✅ **Correct:**

```python
if x > 0 and x < 10:  # Python operators
    print("Valid range")
# or more pythonic:
if 0 < x < 10:
    print("Valid range")
```

Issue: Using && instead of and

Error Type: Range Function Misuse

❌ **Incorrect:**

```python
for i in range(1, 5):
    print(i)  # Prints 1,2,3,4 (not including 5)
```

✅ **Correct (if you want 1,2,3,4,5):**

```python
for i in range(1, 6):  # Include 6 to get 5
    print(i)
```

Issue: range() is exclusive of the end value

Error Type: Modifying List During Iteration

❌ **Incorrect:**

```python
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        numbers.remove(num)  # Modifying during iteration
```

✅ **Correct:**

```python
numbers = [1, 2, 3, 4, 5]
numbers = [num for num in numbers if num % 2 != 0]
# or
for num in numbers[:]:  # Create a copy
    if num % 2 == 0:
        numbers.remove(num)
```

Issue: Modifying list while iterating can skip elements

Error Type: Nested Loop Variable Confusion

❌ **Incorrect:**

```python
for i in range(3):
    for i in range(3):  # Reusing same variable
        print(i, i)
```

✅ **Correct:**

```python
for i in range(3):
    for j in range(3):
        print(i, j)
```

Issue: Using same variable name in nested loops

Functions and Scope

Error Type: Function Definition Syntax

❌ **Incorrect:**

```
python

function calculateArea(length, width):
    return length * width
```

✅ **Correct:**

```
python

def calculateArea(length, width):
    return length * width
```

Issue: Using 'function' keyword instead of 'def'

Error Type: Missing Return Statement

❌ **Incorrect:**

```
python

def get_sum(a, b):
    result = a + b
    # Missing return statement
```

✅ **Correct:**

```
python

def get_sum(a, b):
    result = a + b
    return result
```

Issue: Function returns None if no return statement

Error Type: Global Variable Confusion

❌ **Incorrect:**

```
python
```

```
counter = 0

def increment():
    counter = counter + 1  # UnboundLocalError
    return counter
```

✅ **Correct:**

```
python

counter = 0

def increment():
    global counter
    counter = counter + 1
    return counter
```

Issue: Need 'global' keyword to modify global variables

Error Type: Mutable Default Arguments

❌ **Incorrect:**

```
python

def add_item(item, target_list=[]):
    target_list.append(item)
    return target_list

list1 = add_item("a")
list2 = add_item("b")  # list2 will contain ["a", "b"]
```

✅ **Correct:**

```
python

def add_item(item, target_list=None):
    if target_list is None:
        target_list = []
    target_list.append(item)
    return target_list
```

Issue: Mutable default arguments persist between calls

Error Type: Parameter Order Issues

❌ **Incorrect:**

```python
def greet(greeting="Hello", name):  # Default before non-default
    return f"{greeting}, {name}!"
```

✅ **Correct:**

```python
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"
```

Issue: Default parameters must come after non-default parameters

Error Type: Variable Number of Arguments

❌ **Incorrect:**

```python
def calculate_average(numbers*):
    return sum(numbers) / len(numbers)
```

✅ **Correct:**

```python
def calculate_average(*numbers):
    return sum(numbers) / len(numbers)
```

Issue: *args comes before parameter name

Data Structures

Error Type: List Index Out of Range

❌ **Incorrect:**

```python
```

```
numbers = [1, 2, 3, 4, 5]
print(numbers[5])  # Index 5 doesn't exist
```

✅ **Correct:**

```
python

numbers = [1, 2, 3, 4, 5]
if 5 < len(numbers):
    print(numbers[5])
else:
    print("Index out of range")
# or
print(numbers[4])  # Last element (0-indexed)
```

Issue: Python uses 0-based indexing

Error Type: Dictionary Key Errors

❌ **Incorrect:**

```
python

student = {"name": "Alice", "age": 20}
print(student["grade"])  # Key doesn't exist
```

✅ **Correct:**

```
python

student = {"name": "Alice", "age": 20}
print(student.get("grade", "Not available"))
# or
if "grade" in student:
    print(student["grade"])
```

Issue: Accessing non-existent dictionary key raises KeyError

Error Type: Tuple Modification

❌ **Incorrect:**

```
python
```

```
coordinates = (10, 20)
coordinates[0] = 15  # Tuples are immutable
```

✅ **Correct:**

```
python

coordinates = (10, 20)
coordinates = (15, coordinates[1])  # Create new tuple
# or use list if modification needed
coordinates = [10, 20]
coordinates[0] = 15
```

Issue: Tuples are immutable

Error Type: Set Operations

❌ **Incorrect:**

```
python

numbers = {1, 2, 3, 4, 5}
print(numbers[0])  # Sets don't support indexing
```

✅ **Correct:**

```
python

numbers = {1, 2, 3, 4, 5}
for num in numbers:
    print(num)
    break  # Print first element
# or convert to list
numbers_list = list(numbers)
print(numbers_list[0])
```

Issue: Sets are unordered and don't support indexing

Error Type: List vs String Operations

❌ **Incorrect:**

```
python
```

```
text = "hello"
text.append("!")  # Strings don't have append method
```

✅ **Correct:**

```python
text = "hello"
text = text + "!"  # String concatenation
# or
text_list = list(text)
text_list.append("!")
text = "".join(text_list)
```

Issue: Strings and lists have different methods

Error Type: Shallow vs Deep Copy

❌ **Incorrect:**

```python
original = [[1, 2], [3, 4]]
copy = original
copy[0][0] = 99  # Modifies original too
```

✅ **Correct:**

```python
import copy
original = [[1, 2], [3, 4]]
shallow_copy = original.copy()  # or original[:]
deep_copy = copy.deepcopy(original)
deep_copy[0][0] = 99  # Doesn't affect original
```

Issue: Assignment creates reference, not copy

File Handling

Error Type: File Not Closed

❌ **Incorrect:**

```python
file = open("data.txt", "r")
content = file.read()
print(content)
# File not closed
```

✅ **Correct:**

```python
# Using with statement (recommended)
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File automatically closed

# Or manual close
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Issue: Files should always be closed to free resources

Error Type: File Mode Errors

❌ **Incorrect:**

```python
with open("data.txt", "r") as file:
    file.write("New data")  # Writing to read-only file
```

✅ **Correct:**

```python
```

```
with open("data.txt", "w") as file:
    file.write("New data")
# or append mode
with open("data.txt", "a") as file:
    file.write("Additional data")
```

Issue: File opened in wrong mode for intended operation

Error Type: File Encoding Issues

❌ **Incorrect:**

```
python

with open("data.txt", "r") as file:
    content = file.read()  # May fail with non-ASCII characters
```

✅ **Correct:**

```
python

with open("data.txt", "r", encoding="utf-8") as file:
    content = file.read()
```

Issue: Not specifying encoding can cause issues with special characters

Error Type: Path Handling

❌ **Incorrect:**

```
python

filename = "C:\new_folder\data.txt"  # Escape character issues
```

✅ **Correct:**

```
python
```

```
# Use raw string
filename = r"C:\new_folder\data.txt"
# or forward slashes (works on Windows too)
filename = "C:/new_folder/data.txt"
# or use pathlib (recommended)
from pathlib import Path
filename = Path("C") / "new_folder" / "data.txt"
```

Issue: Backslashes are escape characters in strings

Error Type: File Existence Check

❌ **Incorrect:**

```
python

with open("maybe_exists.txt", "r") as file:
    content = file.read()  # May raise FileNotFoundError
```

✅ **Correct:**

```
python

import os
if os.path.exists("maybe_exists.txt"):
    with open("maybe_exists.txt", "r") as file:
        content = file.read()
else:
    print("File not found")

# or use try-except
try:
    with open("maybe_exists.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("File not found")
```

Issue: Should check file existence or handle exceptions

Object-Oriented Programming

Error Type: Missing self Parameter

❌ Incorrect:

```python
class Student:
    def __init__(name, age):
        name = name
        age = age
```

✅ Correct:

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Issue: Missing 'self' parameter in method definition

Error Type: Accessing Instance Variables

❌ Incorrect:

```python
class Car:
    def __init__(self, brand):
        self.brand = brand

    def display_info(self):
        print(brand)  # Missing self reference
```

✅ Correct:

```python
class Car:
    def __init__(self, brand):
        self.brand = brand

    def display_info(self):
        print(self.brand)
```

Issue: Must use self to access instance variables

Error Type: Class vs Instance Variables

❌ **Incorrect:**

```python
class BankAccount:
    balance = 0  # This is a class variable

    def __init__(self, initial_balance):
        balance = initial_balance  # Not setting instance variable
```

✅ **Correct:**

```python
class BankAccount:
    def __init__(self, initial_balance):
        self.balance = initial_balance  # Instance variable
```

Issue: Confusion between class and instance variables

Error Type: Inheritance Issues

❌ **Incorrect:**

```python
class Animal:
    def __init__(self, name):
        self.name = name

class Dog:  # Missing inheritance
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

✅ **Correct:**

```python
```

```python
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):  # Inherit from Animal
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed
```

Issue: Not properly inheriting and calling parent constructor

Error Type: Method Overriding

❌ **Incorrect:**

python

```python
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area():  # Missing self parameter
        return width * height  # Missing self reference
```

✅ **Correct:**

python

```
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

Issue: Missing self in overridden method

Error Type: Property vs Method

❌ Incorrect:

python

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14 * self.radius ** 2

circle = Circle(5)
print(circle.area())  # Calling property as method
```

✅ Correct:

python

```python
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14 * self.radius ** 2

circle = Circle(5)
print(circle.area)  # Access property without parentheses
```

Issue: Properties are accessed without parentheses

Error Handling and Exceptions

Error Type: Bare Except Clause

❌ **Incorrect:**

```python
try:
    result = 10 / int(input("Enter number: "))
except:  # Catches all exceptions
    print("Error occurred")
```

✅ **Correct:**

```python
try:
    result = 10 / int(input("Enter number: "))
except ValueError:
    print("Invalid input - not a number")
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Issue: Bare except catches all exceptions, including system exits

Error Type: Exception Handling Order

❌ **Incorrect:**

```python
try:
    file = open("data.txt")
    data = file.read()
except Exception as e:  # Too general, comes first
    print("General error")
except FileNotFoundError:  # Specific exception after general
    print("File not found")
```

✅ **Correct:**

```python
try:
    file = open("data.txt")
    data = file.read()
except FileNotFoundError:  # Specific exceptions first
    print("File not found")
except PermissionError:
    print("Permission denied")
except Exception as e:  # General exception last
    print(f"Unexpected error: {e}")
```

Issue: More specific exceptions should come before general ones

Error Type: Finally Block Misuse

❌ **Incorrect:**

```python
try:
    file = open("data.txt")
    data = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    file.close()  # May error if file wasn't opened
```

✅ **Correct:**

```python
file = None
try:
    file = open("data.txt")
    data = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    if file:
        file.close()
```

Issue: Finally block executes even if exception occurred in try

Error Type: Raising Exceptions

❌ **Incorrect:**

```python
def validate_age(age):
    if age < 0:
        raise "Invalid age"  # Raising string instead of exception
```

✅ **Correct:**

```python
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
```

Issue: Should raise exception object, not string

Error Type: Custom Exception Classes

❌ **Incorrect:**

```python
class CustomError:  # Not inheriting from Exception
    def __init__(self, message):
        self.message = message
```

✅ **Correct:**

```python
class CustomError(Exception):
    def __init__(self, message):
        self.message = message
        super().__init__(self.message)
```

Issue: Custom exceptions should inherit from Exception

Modules and Libraries

Error Type: Import Statement Issues

❌ **Incorrect:**

```python
import math.sqrt  # Cannot import specific function this way
result = sqrt(16)
```

✅ **Correct:**

```python
from math import sqrt
result = sqrt(16)
# or
import math
result = math.sqrt(16)
```

Issue: Incorrect import syntax

Error Type: Module Name Conflicts

❌ **Incorrect:**

```python
import math
math = 42  # Overwriting module name
result = math.sqrt(16)  # Error: int has no sqrt method
```

✅ **Correct:**

```python
import math
number = 42  # Use different variable name
result = math.sqrt(16)
```

Issue: Don't reuse module names as variables

Error Type: Circular Imports

❌ **Incorrect:**

```python
# file1.py
import file2
def func1():
    return file2.func2()

# file2.py
import file1  # Circular import
def func2():
    return file1.func1()
```

✅ **Correct:**

```python
# file1.py
def func1():
    import file2  # Import inside function
    return file2.func2()

# file2.py
def func2():
    return "Result from func2"
```

Issue: Avoid circular imports by restructuring or importing inside functions

Error Type: Package Import Issues

❌ **Incorrect:**

```python
import numpy
import pandas
# Heavy imports at top level
```

✅ **Correct:**

```python
# For scripts, consider lazy imports
def analyze_data():
    import numpy as np
    import pandas as pd
    # Use libraries here

# Or standard approach for modules
import numpy as np
import pandas as pd
```

Issue: Heavy imports can slow startup; consider when they're needed

Error Type: Standard Library vs Third-party

❌ **Incorrect:**

```python
# Assuming json is third-party
try:
    import json
except ImportError:
    print("Please install json library")
```

✅ **Correct:**

```python
```

```python
import json  # json is part of standard library
# For third-party libraries:
try:
    import requests
except ImportError:
    print("Please install requests: pip install requests")
```

Issue: Know which libraries are built-in vs need installation

---

Algorithm Implementation

Error Type: Bubble Sort Implementation

❌ **Incorrect:**

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-1):  # Wrong range
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

✅ **Correct:**

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-i-1):  # Correct range
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Issue: Inner loop range doesn't account for sorted elements

Error Type: Binary Search Edge Cases

❌ **Incorrect:**

```python
def binary_search(arr, target):
    left, right = 0, len(arr)  # Wrong right boundary
    while left <= right:
        mid = (left + right) / 2  # Not integer division
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid
        else:
            right = mid
    return -1
```

✅ **Correct:**

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1  # Correct boundaries
    while left <= right:
        mid = (left + right) // 2  # Integer division
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Issue: Wrong boundaries and update logic

Error Type: Recursive Function Without Base Case

❌ **Incorrect:**

```python
def factorial(n):
    return n * factorial(n - 1)  # No base case
```

✅ **Correct:**

```python
```

```python
def factorial(n):
    if n <= 1:  # Base case
        return 1
    return n * factorial(n - 1)
```

Issue: Missing base case leads to infinite recursion

Error Type: List Comprehension Logic

❌ **Incorrect:**

```python
# Find even numbers and square them
numbers = [1, 2, 3, 4, 5, 6]
result = [x**2 for x in numbers if x**2 % 2 == 0]  # Wrong logic
```

✅ **Correct:**

```python
numbers = [1, 2, 3, 4, 5, 6]
result = [x**2 for x in numbers if x % 2 == 0]  # Check original number
```

Issue: Condition checks squared value instead of original

Error Type: Dictionary Comprehension

❌ **Incorrect:**

```python
numbers = [1, 2, 3, 4, 5]
squares = {x: x**2 for x in numbers if x**2 > 10}  # Logic error
```

✅ **Correct:**

```python
numbers = [1, 2, 3, 4, 5]
squares = {x: x**2 for x in numbers if x**2 > 10}  # This is actually correct
# But if you meant numbers > 10:
large_squares = {x: x**2 for x in numbers if x > 3}
```

Issue: Be clear about what condition you're checking

Data Processing and Analysis (2027 Additions)

Error Type: CSV Processing Without Headers

❌ **Incorrect:**

```python
import csv
with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row[0])  # May include header
```

✅ **Correct:**

```python
import csv
with open('data.csv', 'r') as file:
    reader = csv.reader(file)
    header = next(reader)  # Skip header
    for row in reader:
        print(row[0])

# Or use DictReader
with open('data.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row['column_name'])
```

Issue: Not handling header row in CSV files

Error Type: JSON Parsing Errors

❌ **Incorrect:**

```python
```

```python
import json
data = '{"name": "Alice", "age": 25}'
parsed = json.loads(data)
print(parsed['height'])  # Key doesn't exist
```

✅ **Correct:**

```python
import json
data = '{"name": "Alice", "age": 25}'
parsed = json.loads(data)
print(parsed.get('height', 'Not specified'))
# or
if 'height' in parsed:
    print(parsed['height'])
```

Issue: Not checking for key existence in JSON data

Error Type: Data Type Consistency in Lists

❌ **Incorrect:**

```python
data = [1, 2, '3', 4.5, '6']
total = sum(data)  # Type error with strings
```

✅ **Correct:**

```python
```

```
data = [1, 2, '3', 4.5, '6']
# Convert all to float
cleaned_data = []
for item in data:
    try:
        cleaned_data.append(float(item))
    except ValueError:
        print(f"Skipping invalid data: {item}")
total = sum(cleaned_data)

# Or using list comprehension
cleaned_data = [float(x) for x in data if str(x).replace('.','').isdigit()]
```

Issue: Mixed data types prevent numeric operations

Error Type: Large Dataset Memory Issues

❌ **Incorrect:**

```
python

def process_large_file(filename):
    with open(filename, 'r') as f:
        all_data = f.readlines()  # Loads entire file into memory
        for line in all_data:
            process_line(line)
```

** ✅ Correct